Table of contents

Accessing the debug console

Debug ports

BrightScript console (port 8085)...

Cross-component backtrace

Show all...

Debugging code

Testing Roku apps involves using a debug console and access to a variety of ports. The debug console provides a window into the runtime environment and provides features such as crash logs, stack-traces and much more.

Accessing the debug console

The debug console can be accessed using telnet through a shell application such as PuTTY (http://www.putty.org/) for Windows or *terminal* on Mac and Linux:

```
telnet roku-ip-address 8085
```

The console shows the output of your app during runtime. If the app crashes, the debugger will display the line number of the error, as well as the contents of variables at the time of the crash. Compilation errors (e.g. "Syntax Error") are also displayed here. The developer console should be open whenever an app is sideloaded to catch any possible startup errors.

In addition to displaying console output, the shell can also be used as an interactive debugger. When your application is running, simply press `ctrl-C` to break into the application and enter debug mode. You will see a "BrightScript Debugger>" prompt, where you can type commands.

You can also force your app to break at a specific point by inserting `STOP` statements in your code. Be sure to remove this when submitting your app for publication.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Debug ports

Telnet Port	Thread	Description
8080	debug server	debug server containing a host of utilities
8085	BrightScript console	BrightScript runtime environment
8087	Screensaver	The starting point for screensavers

BrightScript console (port 8085) commands

Command	Description
`bsc`	Print current BrightScript component instances
`bscs`	Print a summary of BrightScript component instance counts by component type
`brkd`	Toggle whether BrightScript should break into the debugger after non-fatal diagnostic messages
`bt`	Print backtrace of call function context frames
`classes`	Print BrightScript component classes
`cont or c`	Continue script execution
`down or d`	Move down the, function context chain one
`exit`	Exit shell
`gc`	Run garbage collector
`help`	Print the list of BrightScript console commands
`last or l`	Print the last line that executed
`list`	List current function
`next or n`	Print the next line to execute
`over`	Step over a function
`out`	Step out of a function

Command	Description
`print, p, or ?`	Print a variable or expression
`step, s, or t`	Step one program statement
`threads <ID>` or `ths <ID>`	List all current executed suspended threads
`thread <ID>` `or th <ID>`	Select a suspended thread to debug - all following debug commands will execute within that thread
`up or u`	Move up the function context chain one
`var`	Print local variables and their types/values

BrightScript statements can also be compiled and executed in the console. This can be used to change variables during execution or call a function that prints useful information about the state of your program.

Cross-component backtrace

As of Roku OS 14.6, you can use the `backtrace`, `up`, `down`, `over`, and `out` commands in the debug console on stack frames entered via `callFunc` or an observer callback, in addition to a normal BrightScript function call.

For example, if roSgNode A calls into roSgNode B on the same thread (for example, via CallFunc (/docs/developer-program/core-concepts/handling-application-events.md#functional-fields)) and then B breaks into the call, you can now view the calls belonging to both A and B in the backtrace of the thread.

Type mismatch reporting

As of Roku OS 10.5, the BrightScript debug console provides more specific reporting of "type mismatch" errors to help developers identify and resolve these types of bugs in their code. For example, attempting to evaluate whether an integer value equals a string in an expression (for example, if 12 = "number") results in the following error message: `Type mismatch. Operator "=" cannot be applied to "Integer" and "String"`.

SceneGraph applications

Beginning with Roku OS 7.5 and above, the main BrightScript console (port 8085) provides context for all threads. This eliminates the need to have multiple telnet sessions open for each running thread and ports 8089 - 8093 will no longer be used.

As seen below, any break or `stop` in the app will suspend all threads. All threads will be listed with the following information:

- ID: thread ID
- Location: file the thread originated from and line number
- Source code: current line of code

The current selected thread will be marked with an `*`.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
BrightScript Micro Debugger.
Enter any BrightScript statement, debug commands, or HELP.
Suspending threads...
Thread selected: 0* pkg:/source/Main.brs(19)          msg = wait(0, m.port)
Current Function:
011:     m.port = CreateObject("roMessagePort")
012:     screen.setMessagePort(m.port)
013:
014:     'Create a scene and load /components/helloworld.xml'
015:     scene = screen.CreateScene("HelloWorld")
016:     screen.show()
017:
018:     while(true)
019:*        msg = wait(0, m.port)
020:         msgType = type(msg)
021:         if msgType = "roSGScreenEvent"
022:             if msg.isScreenClosed() then return
023:         end if
Break in 19
019:        msg = wait(0, m.port)
Backtrace:
#0  Function main() As Void
  file/line: pkg:/source/Main.brs(19)
Local Variables:
global          Interface:ifGlobal
m               roAssociativeArray refcnt=2 count:1
screen          bsc:roSGScreen refcnt=1
scene           bsc:roSGNode refcnt=1
msg             <uninitialized>
msgtype         <uninitialized>
Threads:
ID   Location                        Source Code
 0*   pkg:/source/Main.brs(19)            msg = wait(0, port)
 1    ...                Task.brs(25)    msg = wait(0, m.port)
 2    ...                Task.brs(29)    msg = wait(0, m.port)
  *selected
```

This information can be recalled anytime during debugging using the `threads` command.

SceneGraph debug server (port 8080) commands

Command	Description
brightscript_warnings [*num-warnings*]	Sets the maximum number of BrightScript warnings displayed in the de and therefore should be addressed.

Command	Description

chanperf [-r *seconds*]	Prints the current memory and CPU utilization of an app (RAM usage is
	chanperf
	Sending this command with no arguments generates the following out
	`channel: mem=15156KiB{anon=2720,file=12392,shared=44},%cpu=7{use`
	
	chanperf -r *seconds*
	Executes and repeats the chanperf command the specified number of
	repeating command, use the chanperf command with no arguments or
	Calling this command with the seconds parameter set to 10 generates
	`channel: mem=27124KiB{anon=9684,file=17372,shared=68},%cpu=43{u`
	
	The output for the chanperf -r *seconds* command is sent to port 808
	
	Error message
	If the app is not running, or if undefined attribute is missing from the m
	undefined
	
	You can download a sample app (https://github.com/rokudev/sgnod
	command.

logrendezvous [on | off]	Enable console logging of thread rendezvous. Set to off to disable.
loaded_textures	Displays the current set of images loaded into texture memory.
r2d2_bitmaps	Prints a list of assets loaded into texture memory and the amount of fr respectively. Starting with Roku OS 9.3, the name of each bitmap is incl
remove_plugin *app id*	Removes the indicated app from the local device, as well as from all dev a *app id* of "987654_cf9a", then the following command would remove i The list of available app ids can be seen in the second (from leftmost) c command. The local device must be linked to a Roku account. To use this command, the local device must be linked to a Roku account with the Streaming Store (for example, via an automatic check for upda

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Command	Description
sgnodes all	Prints every existing node created by the currently running app. As of Roku OS 14.5, you can use this command on your published app if generate the app's package file. As of Roku OS 10.0, this prints the number of osref references to the nc app). The bcsref count includes references from "m." variable and local bscref counts. The osref count also includes child references and references from Roku parent, the parent will count as one osref on the child. Additionally, any each node referenced from within that field. These could be in variables component global m or an associative array field of a node. The reported osref count may vary from release to release of Roku OS; of items that the count includes. The bscref count provides a more rele controls. The `sgnodes all`, `sgnodes roots`, and `sgnodes node_ID` command getAllMeta() ifSGNodeChildren (/docs/references/brightscript/interfac SceneGraph node.
sgnodes roots	Prints every existing node without a parent created by the currently ru are being kept alive by direct BrightScript references. These could be in including a component global m or an associative array field of a node.
sgnodes node_ID	Prints nodes with an id field set to node_ID, except it, bypasses all the h the order of node creation. It will list multiple nodes if there are several
sgperf start|clear|report|stop	Provides basic node operation performance metrics. This command tra an operation on an existing node, and whether it involves a rendezvous zero, report - prints current counts with rendezvous as a percentage, st
sgversion force or default 1.0 or 1.1	Changes the observer callback model and overrides the default rsg_ver will set rsg_version=1.0 regardless of what is specified in the manifest. specified in the manifest. Changing the rsg_version will require restarti Support for the “rsg_version=1.0” manifest flag is deprecated. This depr thus should not be expected to work). All apps must adopt the current
fps_display	Displays frames-per-second and free memory on-screen. Leverage this the fps meter: fps_display 1 turns on the fps meter. It presents a 1-sec turns the meter back off.
free	Provides a snapshot of the amount of in-use and free memory on the c

Troubleshooting common development errors

There are several very common errors that you will encounter when developing SceneGraph apps. Quite often these errors are caused by not spelling component names or variables correctly but may appear as different types of errors on the display screen and in the debugger.

Graphic image does not appear, question mark appears instead of image

The graphic image file was not found in the location specified in the application. Check that graphic image file is in the specified location, either on your development server, or in the application ZIP package, usually in the pkg:/images directory. Make sure the path to the file is correct, and the name of the file is spelled correctly. Roku SceneGraph applications, like previous Roku applications, generally follow the convention used in many client display applications, such as web browsers, which is to show a default image if the specified image cannot be found. If a question mark image is shown in a Roku SceneGraph app, check the path and file name spelling to ensure that the correct graphic image appears.

Debugger message: "Type Mismatch"

This usually means that a BrightScript variable has been incorrectly spelled after it has been declared, never declared at all, or declared as a local variable in another function. Check the backtrace information supplied by the debugger for the local variables used at the time of the error, and note that the variables listed as . Check for the declaration of the variable or variables listed earlier in the application prior to the error message, and correct the spelling, or declare them correctly, as needed. In many cases, the error occurs because there is an attempt to use a local variable declared in one function block in another function. To correct this, you can declare the variable using the m object reference which gives the variable file scope.

Debugger message: "'Dot' Operator attempted with invalid BrightScript Component or interface reference"

This message will often coincide with a blank screen. The line number at which the error is detected will be flagged with an asterisk, and the message will provide the name of the file in which the error was detected:

```
020:*      smallexamplesize = smallexample.localBoundingRect()
...
'Dot' Operator attempted with invalid BrightScript Component or interface reference. (runtime error &h
020:      smallexamplesize = smallexample.localBoundingRect()
```

Roku (https://developer.roku.com/develop) Developer
(https://developer.roku.com/develop)

This message will appear if a component by that name has either not been created, or an attempt is made to access a component member using an incorrectly spelled component name. Check the backtrace information supplied by the debugger for the component objects and variables used at the time of the error, and note the component objects listed as invalid:

```
Backtrace:
#0  Function init() As Void
  file/line: ...pkg:/components/smallexamplescene.xml(20)
Local Variables:
global          rotINTERFACE:ifGlobal
m               roAssociativeArray refcnt=3 count:2
devinfo         bsc:roDeviceInfo refcnt=1
screenresolution roAssociativeArray refcnt=1 count:3
smallexample     Invalid
smallexamplesize <uninitialized>
centerx         <uninitialized>
centery         <uninitialized>
```

Note also the variables that were assigned values from interface functions on invalid component objects will be listed as . Typically in Roku SceneGraph applications, the problem is caused by attempting to create a component object for a component class name that is not in either the built-in node classes, or extended node classes declared in the application package components directory. To fix this error, scroll up in the debugger output to the point at which the component object creation error occurred, which will have the following error message:

```
BRIGHTSCRIPT: ERROR: roSGNode: Failed to create roSGNode with type Rectangleexample: ...pkg:/comp
```

This shows the file and line number where the actual component object creation error occurred. To fix the error, correct the mismatch between the component name and the component object creation function argument. Quite often this is the result of a case mismatch between the extended component name in a element, since these names are case-sensitive. Correct the case-sensitive spelling of the component name either in the component file, or at the point where you attempted to create the component object.

List or grid fails to appear, or first item is blank or missing information

This often indicates that the ContentNode assigned to the content field of the list or grid either does not exist, or was assigned after focus was set on the list or grid. Ensure that the ContentNode has been created successfully at the time it is assigned to the list or grid content field. Then check that focus was set on the list or grid after the content field is assigned a valid ContentNode. Since you will generally be generating a ContentNode by parsing data from an XML or JSON file downloaded from your server in a Task node (or possibly downloaded as "singleton" at the time the SceneGraph app was created in the main.brs file and converted), make sure you set the content field and focus on the list or grid in this way:

Roku (https://developer.roku.com/develop) Developer
(https://developer.roku.com/develop)

```
sub showvideolist()
  m.videolist.content = m.readVideoContentTask.videocontent
  m.videolist.setFocus(true)
end sub
```

This is a typical callback function that is triggered by the ContentNode being created in a Task node (in this case, the event was a new ContentNode assigned to the m.readVideoContentTask Task node object element videocontent field).

Also, if you are having problems with a callback function not assigning a valid ContentNode, carefully check that the field observers were set before the Task node was configured and launched (but after the Task node object was created). For example, for the above example, the Task node object should have been created, had the field observers set, configured, and launched, in that order:

```
m.readVideoContentTask = createObject("RoSGNode","MetaDataCR")
m.readVideoContentTask.observeField("videocontent","showvideolist")
m.readVideoContentTask.metadatauri = "pkg:/server/videometadata.xml"
m.readVideoContentTask.control = "RUN"
```

This is the sequence to use for all object field observers. If an event in an observed field triggers a callback function after the field observer is set, the callback function is likely not to work as expected.

Debugging SceneGraph node memory usage

In a development app, you can use the special SceneGraph debugging commands to evaluate node memory usage, including potential memory leaks.

A node leak in an app is simply any node that is no longer reachable by the app code, yet still exists. Such a node is holding onto memory and other resources unnecessarily. Due to reference counting of nodes in BrightScript, the only way to create true leaks is to create isolated cycles by severing external references to objects which refer to each other. For node cycles, that means that either a node references one of its ancestors (including itself) in a field or one or more nodes reference each other in fields. Note that field types that can hold references to nodes include node, associative array, and regular array.

Some nodes are not leaks since they are still reachable by the code, but they are nevertheless unneeded if there is no intent for the code to use them ever again.

The following ifSGNodeChildren methods can be helpful to find both leaked and unneeded nodes.

The getRoots() method can be used to find nodes and node cycles that are not referenced by any other nodes, and these roots should be inspected carefully to determine whether any of the root nodes are leaks or unneeded.

If they are leaks, they must be part of cycles or they would have been deleted already. To fix these, the code should either not create the cycle or break the cycle before removing the last code reference to the node. Automatic garbage collection will recover these leaks after the app exits, but a long-running app accumulating leaks may begin to slow down or eventually even crash.

If the nodes are merely unneeded, they might not be in cycles but may be simply stored somewhere the code will no longer access even though it could. If so, the app should be updated to dereference any nodes that are no longer needed. This can be done by setting the variables or fields that are holding the nodes to invalid.

The getAll() and getAllMeta() methods can also be used to find nodes that are still referenced by other nodes, but still may be leaks or unneeded. The set of top-level nodes directly under the <All_Nodes> XML element is similar to those returned by the getRoots() method, and any leaked cycle will have at least one member in this set. Unneeded or cyclically linked nodes can appear anywhere in the node hierarchy, however, so the complete listing should be reviewed when leaks are suspected. A leaf entry that references any node above it in a tree represents a cycle.

Note that determining whether a node is unneeded is not always a simple task, and is up to the intent of the app code. Outright cyclical leaks can be detected automatically, but since that can be a time consuming operation, it is only performed on app exit, and the app should be coded to avoid creating them.

You can also use the equivalent sgnodes debugger commands (see Special SceneGraph debugging commands).

Debugging video lesson

You can learn how to debug apps by watching the Debugging channels (/videos/courses/rsg/debugging.md) video lesson in Roku's SceneGraph: Build a Channel online video course (https://developer.roku.com/videos/courses/rsg/overview.md).

This lesson describes how to view the crash logs, stack traces, and app performance metrics in the console. And it guides you on how to use the interactive debugger to type commands for checking and updating variable values, calling functions, and stepping through the application.

Next

B2B Privacy Notice (https://docs.roku.com/published/b2bprivacypolicy/en/us)

Developer Site (https://developer.roku.com)

About ads & cookies (https://docs.roku.com/published/cookiepolicy)

My Privacy Choices
(https://privacy.roku.com/ccpa)
Developer Terms & Agreements (https://developer.roku.com/docs/features/legal/developer-terms.md)

CA B2B Privacy Notice

© 2008-2026 Roku, Inc. (https://www.roku.com)